

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: COMPUTER NETWORKS
COURSE CODE: CSA07

COURSE OUTCOME COVERED

CO5: Measure network performance using key metrics with simulation tools.
(BL4,BL5)

Assessment Tool Weightage: 30%

Annexure A

Pseudocode Writing Task (Algorithm Design)

1. In a network simulation using Cisco Packet Tracer, a student records total data received as 8000 bits in 4 seconds and observes that 200 packets were sent while 180 packets were received. Write a pseudocode algorithm to calculate the network throughput in bits per second and the packet loss percentage based on the given values. The algorithm should include validation to ensure that time and total packets sent are not zero to avoid errors. It should display both the throughput and packet loss results clearly. Add logic to classify the network as Efficient if throughput is greater than 1500 bps and packet loss is less than 10%, otherwise classify it as Inefficient. Finally, ensure the algorithm outputs a complete summary of the network performance based on the calculated metrics.
2. An organization has multiple network links connecting its branches. The administrator wants to monitor bandwidth utilization and identify overloaded links. Write a pseudocode algorithm that periodically collects the bandwidth usage of each network link and stores the values in an array or list. Use a loop to continuously monitor all links. Add conditions to detect when bandwidth utilization exceeds 80% and recommend switching traffic to a backup link. Explain how this algorithm supports efficient bandwidth management and prevents network congestion.
3. A company has six branch offices connected through multiple network links. Each link has different bandwidth, delay, and utilization values, which change throughout the day. The network administrator wants to continuously identify the best communication path between the headquarters and each branch based

on overall link quality rather than just the shortest distance. Write a pseudocode algorithm that periodically collects the bandwidth, delay, and utilization of every network link and stores the values in suitable arrays or lists. Use a loop to repeat the monitoring process at regular intervals. Calculate a link quality score for each path using the collected metrics and select the path with the highest score. Include conditions to detect when a link becomes overloaded or fails, automatically selecting an alternate path and generating an alert for the administrator. Explain how your algorithm improves network reliability, routing efficiency, and fault tolerance while adapting to changing network conditions.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

1. Throughput & Packet Loss Calculator

Given: total data received = 8000 bits, time = 4 seconds, packets sent = 200, packets received = 180.

Pseudocode

```
BEGIN
  INPUT total_bits_received, time_seconds, packets_sent, packets_received

  // --- Validation ---
  IF time_seconds <= 0 THEN
    PRINT "Error: Time must be greater than zero."
    STOP
  END IF

  IF packets_sent <= 0 THEN
    PRINT "Error: Total packets sent must be greater than zero."
    STOP
  END IF

  // --- Throughput calculation ---
  throughput_bps = total_bits_received / time_seconds

  // --- Packet loss calculation ---
  packets_lost = packets_sent - packets_received
  packet_loss_percent = (packets_lost / packets_sent) * 100

  // --- Classification ---
  IF throughput_bps > 1500 AND packet_loss_percent < 10 THEN
    status = "Efficient"
  ELSE
    status = "Inefficient"
  END IF

  // --- Output summary ---
  PRINT "----- NETWORK PERFORMANCE SUMMARY -----"
  PRINT "Throughput: ", throughput_bps, " bps"
  PRINT "Packet Loss: ", packet_loss_percent, "%"
  PRINT "Network Status: ", status
  PRINT "-----"
END
```

Worked Trace with Given Values

- $\text{throughput_bps} = 8000 / 4 = 2000 \text{ bps}$
- $\text{packets_lost} = 200 - 180 = 20 \rightarrow \text{packet_loss_percent} = (20/200) \times 100 = 10\%$
- Classification check: throughput (2000) > 1500 ✓, but packet_loss (10%) is NOT < 10% (it equals 10, fails the strict '<') → status = Inefficient

Output: Throughput = 2000 bps, Packet Loss = 10%, Network Status = Inefficient.

2. Bandwidth Utilization Monitor

Pseudocode

```
BEGIN
  DECLARE link_names[]           // e.g., ["Link-1", "Link-2", ..., "Link-N"]
  DECLARE bandwidth_usage[]      // parallel array, % utilization per link
  DECLARE backup_link_map{}      // maps each link to its designated backup
  link

  INPUT link_names, backup_link_map

  WHILE monitoring_active == TRUE DO      // continuous monitoring loop
    FOR i = 0 TO length(link_names) - 1 DO
      current_usage = GET_CURRENT_BANDWIDTH_USAGE(link_names[i])
      bandwidth_usage[i] = current_usage    // store/update reading

      PRINT link_names[i], ": ", current_usage, "% utilized"

      IF current_usage > 80 THEN
        PRINT "ALERT: ", link_names[i], " is overloaded (",
current_usage, "%)."
        backup = backup_link_map[link_names[i]]
        IF backup IS NOT NULL THEN
          PRINT "Recommend switching traffic to backup link: ", backup
        ELSE
          PRINT "WARNING: No backup link configured for ",
link_names[i]
        END IF
      END IF
    END FOR

    WAIT(monitored_interval)      // e.g., pause 60 seconds before next
cycle
  END WHILE
END
```

How This Supports Bandwidth Management and Prevents Congestion

- Continuous, periodic monitoring (the outer WHILE loop) means overload conditions are caught within one monitoring interval rather than being discovered only after users report slowness.
- Storing each link's usage in an array/list every cycle keeps a live utilization picture across all links simultaneously, letting the administrator (or an automated system) compare links and spot patterns (e.g., one link consistently near capacity at certain hours).
- The 80% threshold check triggers proactive action before a link is fully saturated — switching to a backup link at 80% avoids the packet queuing, delay, and drops that would occur if the link were allowed to reach 100% utilization.
- Automating the recommendation (rather than requiring manual link-by-link inspection) reduces the time between overload onset and corrective action, directly reducing the window during which congestion can affect users.

3. Best-Path Selection with Link Quality Scoring

Pseudocode

```
BEGIN
  DECLARE branches[]           // e.g., ["Branch-1", ..., "Branch-6"]
  DECLARE paths{}              // paths[branch] = list of candidate
links to that branch
  DECLARE bandwidth[], delay[], utilization[] // parallel arrays per link,
refreshed each cycle
  DECLARE current_best_path{}   // current_best_path[branch] =
currently selected link

  INPUT branches, paths

  WHILE monitoring_active == TRUE DO
    FOR EACH branch IN branches DO
      best_score = -INFINITY
      best_link = NULL

      FOR EACH link IN paths[branch] DO
        b = GET_BANDWIDTH(link)
        d = GET_DELAY(link)
        u = GET_UTILIZATION(link)
        store b, d, u into bandwidth[], delay[], utilization[] for this
link

        // Higher bandwidth is good; higher delay/utilization is bad.
        // Normalise so all three terms contribute on a comparable 0-1
scale.
        link_quality_score = (W1 * NORMALIZE(b))
                             - (W2 * NORMALIZE(d))
                             - (W3 * NORMALIZE(u))
        // e.g., W1 = 0.4, W2 = 0.3, W3 = 0.3 (weights sum to 1, tunable)

        IF is_overloaded(u) OR has_failed(link) THEN
          PRINT "ALERT: Link ", link, " to ", branch, " is
overloaded/failed."
          CONTINUE // skip scoring this link this
cycle

        END IF

        IF link_quality_score > best_score THEN
          best_score = link_quality_score
          best_link = link
        END IF
      END FOR

      IF best_link != current_best_path[branch] THEN
        PRINT "Switching path for ", branch, " to ", best_link,
          " (score = ", best_score, ")"
        current_best_path[branch] = best_link
      END IF
    END FOR
  END WHILE
```

```

        IF best_link == NULL THEN
            PRINT "CRITICAL ALERT: No viable path available to ", branch, "!"
        END IF
    END FOR

    WAIT(monitoring_interval)
END WHILE
END

```

How This Improves Reliability, Routing Efficiency, and Fault Tolerance

- **Reliability:** by continuously re-scoring every candidate link on bandwidth, delay, AND utilization together (not distance alone), the algorithm routes traffic over links that are actually performing well right now, not just the topologically shortest one — directly addressing the requirement to select paths on 'overall link quality'.
- **Routing efficiency:** the weighted link_quality_score lets the administrator tune W1/W2/W3 to the organisation's priorities (e.g., weight delay more heavily for VoIP-heavy branches), so the 'best' path adapts to what actually matters for that traffic, rather than a one-size-fits-all shortest-path metric.
- **Fault tolerance:** the is_overloaded/has_failed check removes a failing or saturated link from consideration entirely before scoring, and the algorithm automatically re-selects the next-best remaining link (best_link) and raises an alert — this happens every monitoring cycle, so recovery from a link failure is automatic and bounded by the monitoring interval, not dependent on manual administrator intervention.
- **Adapting to changing conditions:** because the WHILE loop re-collects bandwidth/delay/utilization every cycle rather than computing the best path once, the chosen path naturally shifts over the course of a day as link conditions change (e.g., congestion during business hours), keeping the headquarters-to-branch route continuously optimised rather than statically fixed.